

AgentWorkShop: Node-Native Binding and Governed Write Paths for Agent Teams in Industrial Supervision

Anonymous Authors

Abstract—Bringing language-model agents into industrial supervisory control requires associating the semantics, authority, and history of each command across heterogeneous devices and multiple agents. We present AGENTWORKSHOP, a node-native framework in which explicit member-to-node bindings determine each agent’s tools, source-dependent checks govern agent-issued writes, and attributable records with an independent backstop keep authorization, device execution, and process judgment distinguishable. A scripted channel layer and a fixed controller drive the governed write path; the LLM decision path stays disabled. Results come from one archive over three simulated scenarios with real protocol transports, including a 49-signal biaxial-film line. The run passes 75 of 75 checks; the paper also accounts what the gate does not see: task-level failures surface as warnings, three checks carry skipped write sub-checks, and the suite is not deterministic at the mission layer. The writable stacks reject 24/24 predefined gross out-of-constraint probes and none of four legal writes; a second scenario rejects 9/9 without false blocks. A fixed non-LLM controller reaches 0.963–0.972 of an offline grid-search reference. A scripted team mission brings biaxial-film thickness from 28.10 to 25.68 μm (objective $25.0 \pm 0.7 \mu\text{m}$) in three governed writes on two protocols; seven of the ten reportable runs (of eleven same-seed archives) end in the band (small- n); none of the collapses trips the hard gate. These results characterize the governance path on simulated plants; physical validation and controlled agent comparisons remain open.

Index Terms—heterogeneous protocols, human-in-the-loop, industrial informatics, large language models, multi-agent systems, supervisory control.

I. INTRODUCTION

LARGE language models (LLMs) are being explored for industrial diagnosis, engineering assistance, and control-program generation [1], [2], [3], [4]. Recent systems also connect agents to simulated controllers or real machines [5], [6]. For a production-line deployment, the engineering problem is not whether an LLM can issue a command but how to preserve its meaning, authority, and history across heterogeneous equipment, multiple agents, and changing operating recipes.

Consider a team correcting film thickness within pressure and temperature limits: one member inspects telemetry, another proposes a setpoint, and a lead coordinates the task. A protocol connection alone does not identify which member may change which node, which recipe applies to the current batch, whether a human must approve, or whether the subsequent observation measures the commanded register or the process response. Implementing these associations separately for each application duplicates configuration and makes failures hard to reconstruct; conversely, one shared interface does

not by itself establish atomic execution, physical safety, or valid optimization. The framework below contrasts these two patterns: advisory, application-specific integration on one side, and explicit binding over shared production context on the other, with readback and attribution attached to every governed write.

We present AGENTWORKSHOP, a configurable framework for this supervisory integration problem: an industrial node combines its acquisition or setpoint interface with engineering semantics, and an explicit binding grants a deployed channel member access to that node; a line/product/recipe/run model supplies the production context shared by observations and actions. The platform core hosts agent orchestration and industrial supervision together, while engine subprocesses, optional external stores, and field endpoints retain their own boundaries. This organization reduces application-specific routing within the supported driver set without equating shared state with an end-to-end transaction.

The agent loop operates at supervisory, seconds-to-minutes timescales. Manual bindings require an approval decision; automatic bindings omit that wait but retain the applicable execution checks; write attribution and separately triggered compensation make a failed proposal, a failed write, and an unsuccessful adjustment inspectable as different events.

The contributions are threefold:

- 1) A **node-native integration model** connecting heterogeneous acquisition and control nodes, deployed agent teams, and versioned production runs through explicit capabilities and shared identifiers (Sec. III).
- 2) An **implemented supervisory workflow** that separates team coordination, source-dependent write checks, agent-submitted judgments, physical compensation, and human decisions; its seven mechanisms and the cross-cutting human-in-the-loop (HITL) boundary are described with their actual scopes and exceptions (Sec. IV).
- 3) A **traceable, layered evaluation** anchored on one reference archive detailed in Sec. V-A and an eleven-archive rerun family sharing its seed (Sec. V-E). All 75 suite checks pass, and the writable stacks reject every predefined out-of-constraint probe while blocking none of the designated legal writes. A fixed non-LLM controller reaches $J/J^* = 0.963\text{--}0.972$ of an offline reference over three seeds, and a scripted team mission brings a simulated film line from 28.10 to 25.68 μm with three governed writes on two protocols; seven of the ten reportable runs attain the objective band (six of nine

excluding the reference). All evidence is simulation-tier and characterizes the platform paths, not agent quality.

These contributions correspond to the evaluation's evidence layers (Sec. V, Table I), each layer reported separately rather than pooled into one score. The paper specifies an integration model, implements it end to end, and reports what that implementation does and does not guarantee; it proposes no new control law, learning method, or safety certification argument, and claims configuration reuse within the existing driver and scenario vocabulary, not certified safety, general industrial autonomy, or LLM-controller superiority.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based distributed manufacturing control has a two-decade history [7]; what is contributed here is the binding of LLM-team membership to node capability. Industrial LLM applications span industrial knowledge-graph construction [8] and diagnosis [9], PLC program generation and verification [3], [4], and digital-twin configuration [10]. Not all industrial agents are advisory: natural-language interfaces have controlled machines over OPC UA [6], LLM agents have been studied in simulated fault-tolerant control and digital-twin-supported fault handling [5], [11], and InstructMPC injects language-derived context into model-predictive control [12]. Closest to our setting, agent-based supervision has been studied for service-oriented industrial cyber-physical systems [13], and LLM-based multi-agent systems coordinate shopfloor manufacturing tasks [14]; the former assigns agents to services for monitoring and reconfiguration rather than granting member-instance write authority, and neither applies source-dependent execution checks or line/product/recipe/run attribution.

B. Orchestration, Industrial Models, and Execution Governance

AutoGen, LangGraph, and CrewAI organize agent tasks and tool use [15], [16], [17], and Model Context Protocol (MCP) and Agent-to-Agent (A2A) standardize tool and inter-agent interfaces [18], [19]; industrial integration additionally requires node semantics, driver behavior, and operating context. We use a line/product/recipe/run organization informed by ISA-95 and ISA-88 [20], [21]: line and product correspond to level-3 production work units and product definitions, run to production-performance records, and recipe to the ISA-88 master-recipe idea with grade and operating-point semantics; we do not implement the complete procedural models of either standard. The device-twin terminology presumes manufacturing digital twins whose assets carry structured semantic descriptions [22], [23], and the node semantic card is analogous to an Asset Administration Shell submodel [24]; IIoT-gateway digital twins similarly wrap legacy equipment behind semantic abstraction [25], and what is added here is member-level write governance over that abstraction. A goal-driven unit's observe-hypothesize-write-re-observe cycle likewise resembles autonomous chemical research with

LLMs [26]; our writes are governed industrial setpoints rather than self-directed experiment planning.

The ingredients of a governed write—node-scoped permission, range validation, approval, and an audit trail—have counterparts in mature DCS/SCADA/MES practice, typically enforced as role- and application-level controls under IEC 62443-3-3 [27]. LLM-mediated writes additionally require member-instance-level binding, semantic linkage between a write and the verdict or optimization record that produced it, and attribution across engine boundaries. Hard action shielding and control-barrier methods enforce formally specified admissible actions [28], [29]; the rollback manager and rule-based backstop are engineering instances of runtime assurance and simplex-style architectures [30], and are narrower implementation-level checks rather than substitutes for certified plant protection. Human supervision also requires a clear division of responsibility, not merely an approval dialog [31], [32].

C. Evaluation of Tools and Industrial Control

AgentBench and consequential tool-use benchmarks evaluate agent behavior and policy compliance [33], [34]. Industrial datasets such as the Tennessee Eastman Process (TEP) and Secure Water Treatment (SWaT) support process-monitoring research [35], [36], but replay is not evidence of physical actuation, so we separate source checks, mock-driver probes, software-protocol integration, fixed-controller optimization, and exploratory LLM runs. The threat model of indirect prompt injection [37], since formalized and benchmarked together with defenses [38]—a compromised agent writing through legitimate tools—is what the write-side checks are designed to contain, although our evaluation includes no adversarial LLM probes.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP integrates multi-agent reasoning with industrial supervision around a common representation of production assets and operating context. Its organizing object is the node: an acquisition or control endpoint with engineering semantics, device connections, line membership, and agent bindings. A channel organizes the agents that interpret these endpoints, controllers mediate plant interactions, and Fig. 1 locates these components and execution boundaries. The platform also carries subsystems outside this paper's scope, including an automated-modeling laboratory and a three-dimensional twin view; this paper describes the supervision path that the evaluation exercises.

A. A Shared Supervisory Core

The Node.js core hosts channel routing, task scheduling, industrial services, protocol adapters, and platform interfaces: REST and WebSocket endpoints serve operator applications, MCP and A2A interfaces support external integration, and a plugin host extends platform capabilities. These surfaces reuse platform objects, but their exposed operations and permission checks differ, and an agent's industrial authority

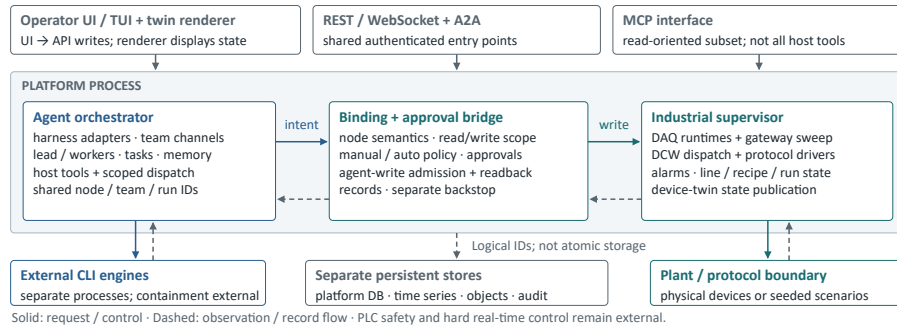


Fig. 1. System architecture and execution boundaries. The co-resident platform process hosts the agent orchestrator, the binding-and-approval bridge, and the industrial supervisor; operator entry points sit above it, and external CLI engines (including the bundled OMP engine), persistent stores, and the plant boundary sit outside it. Shared identifiers support attribution across the logical data model; they do not make storage or physical actuation transactional.

derives from its node bindings, not from channel access or protocol-endpoint availability. CLI engines, including the platform's bundled OMP engine, execute in spawned processes, TimescaleDB and MinIO are external, and connection establishment, queued protocol operations, and readback introduce asynchronous boundaries. Fast regulation and protection therefore remain with the plant controller and its safety equipment.

B. Reasoning, Plant Interaction, and Operational Context

On the reasoning side, a channel combines a lead, worker instances, a task tree, messages, and retrievable memory. Engine adapters allow different reasoning implementations to participate without changing a bound node’s identity or the controller interface.

On the plant side, acquisition nodes turn protocol readings into engineering quantities, control nodes (data-control-write, DCW) translate engineering setpoints into device commands and readback results, per-node runtimes manage timing and in-flight operations, and controller services supply gateway state, recipe context, and write bookkeeping. Nodes and devices have many-to-many configuration associations, but association is distinct from execution routing: a control-node write invokes its configured driver and connection settings rather than iterating over its associated devices, so the interface follows process meaning without implying multi-device actuation by one write.

C. Logical Continuity Across a Production Run

Lines, products, recipes, and runs provide the context in which measurements and interventions become comparable. Their identifiers connect platform state, time series, image frames, operational events, and optimization records across heterogeneous stores: SQLite and file-backed repositories for platform state, TimescaleDB for measurements, MinIO for objects. These constitute one logical data model rather than one transaction domain.

Operators participate through approvals, line operations, and monitoring. Section IV details these mechanisms and marks where their guarantees end.

IV. CORE MECHANISMS

Seven mechanisms carry the integration model (Secs. IV-A–IV-G), plus the cross-cutting HITL boundary (Sec. IV-H); Fig. 2 previews the closed loop they form.

A. Node Connection and Creation

A node separates the meaning of an industrial variable from its transport address. Commissioning binds device integration information to an acquisition or control configuration, tests connectivity, and supplies units, calibration, limits, and production-line membership. Drivers support Modbus TCP, Modbus RTU over TCP, OPC UA [39], MQTT, HTTP/REST, plus a mock implementation for testing; the RTU driver exposes no writable setpoint in the reference scenarios. Creation is not authorization to operate: operator APIs and agent-facing paths carry different permission conditions, and an industrial-tool binding refers to a concrete channel-member instance, not a reusable template, so deploying a template elsewhere does not transfer the binding—a design consequence rather than a measured result, with binding denial exercised in the archive only on the parameter layer.

The semantic card presents the node’s physical quantity, unit, precision, range, current value, freshness, and available recipe context. Keeping a setpoint change within 2 % of span is prompt guidance, not a controller-enforced step limit: the biaxial mission’s first write moves 3.6 % of span and nothing rejects it. Per-node acquisition runtimes manage due times and in-flight sampling, and a rotating gateway sweep apportions that work across intervals, supporting supervisory observation rather than hard real-time sampling.

B. The Agent Channel: Team, Supervision, and Memory

A channel contains a lead, worker instances, a task tree, and message history. Routing resolves recipients within the channel and loads runtimes as needed. The task tree distinguishes submitted from completed work, but those states describe collaboration progress, not device execution: a completed task is not permission to write and not evidence that a write occurred. The scheduler combines periodic supervision with event wake-up and idle backoff, and deterministic rules handle dispatch, retry, and stalled work; scheduler progress cannot resolve

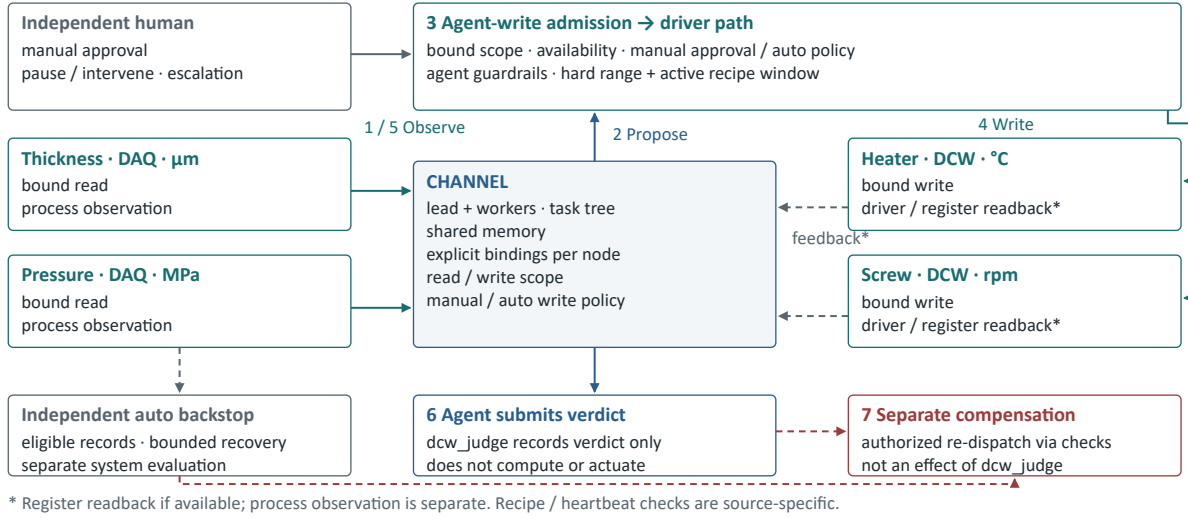


Fig. 2. The closed loop formed by the Section IV mechanisms. Numbered items are loop steps (steps 1 and 5 share the observation arrows), not the seven mechanism subsections of this section. The channel coordinates agents that observe bound nodes, propose setpoints, and pass admission checks before write and readback. Step 6 is the agent's own verdict, the backstop a separate judgment path, and step 7 compensation that neither path performs. `dcw_judge` records a verdict only: it neither computes the conclusion from telemetry nor executes any action.

uncertainty about a plant response, which requires observations and an explicit judgment. Channel memory preserves experience for later retrieval, so a later worker recovers earlier rationale, but a retrieved memory is contextual evidence, not an authorization mechanism or an executable safety specification. Industrial tools resolve the invoking member's bindings at request time, on the judge and rollback paths as well as the write path; the parameter-layer check `agent-param-tools` in Sec. V-C exercises that denial, only for the parameter layer and with a scripted caller.

C. Node Control Execution

Let $S_n = [l_n, u_n]$ be the node range, r the recipe identifier, $W_{r,n}$ the parameter window that recipe r prescribes for node n , q the optional recipe-run identifier supplied with a request, and P_r the nodes with a parameter in that recipe. The effective constraint at controller entry is

$$C(n, r, q) = \begin{cases} S_n \cap W_{r,n}, & r \neq \emptyset, q = \emptyset, n \in P_r, \\ S_n, & \text{otherwise.} \end{cases} \quad (1)$$

An unspecified window endpoint is unbounded. Equation (1) describes normal controller requests, not every low-level executor invocation: recipe application supplies q and so bypasses the active-window branch, hold heartbeats call the executor directly while sharing write exclusion, and ownership and cooldown checks depend on source metadata and are not applied universally.

Calibration maps engineering values to the device domain and readback into physical units; the runtime's tolerance is

$$\tau_n = \max(0.5 \cdot 10^{-d_n}, 0.005(u_n - l_n)), \quad |v_{rb} - v| \leq \tau_n, \quad (2)$$

with display precision d_n , converted to the absolute scale before comparison for linear calibration. A successful readback supports agreement with the commanded value, not proof

that the process reached its desired operating condition, so communication failure and readback mismatch stay distinct from process-effect judgment. With no readback available, displaying the commanded value as a fallback does not establish measured agreement.

The ingress is sequential: binding resolution, with the binding rechecked after any approval; a pause and node-disable gate; agent ownership and directional cooldown; the effective range and window of Eq. (1); then node write exclusion, connection, write, and readback. Asynchronous connection and protocol queuing separate the checks from the device action, so no co-residency or node write exclusion supplies check-act atomicity against a changing recipe state. A successful write triggers journal anchors and, for relevant sources, optimization records; bookkeeping errors are caught without reversing the result, so attribution is traceability, not an atomic commitment across storage and the device.

D. Production-Line Operations

Line operations associate products, recipes, and runs with the nodes that generate measurements or accept setpoints. The run gives a batch identity, so a temperature observed under one product's recipe should not become evidence about another product's operating window. Starting a line validates its recipe and product, creates a run, installs it as active, and writes its parameters individually; an active run is not a certificate that every parameter applied. Acquisition is gated by active batches, so opening a run establishes the context in which measurements are collected; stopping the line clears that context, closes up to 500 open optimization records through the current stop hook, and requests acquisition-offline status. Closure beyond that limit is not guaranteed, and the operation is a supervisory batch command, not a PLC emergency stop.

Monitoring combines live values, node conditions, alarms, and events with historical measurements and frames; the

device-twin scene renders these states without issuing protocol commands, although the operator console can submit API writes. Neither the scene nor a stopped batch is a physical safety barrier; between batches the run gate limits acquisition while plant-side protection remains independent.

E. Agent Closed-Loop Control

The feedback sequence is observation, hypothesis, intervention, judgment, and retention or recovery. Fig. 2 shows this closed loop over the bound nodes of a line. The agent inspects bound-node context and acquisition history, states an expected effect, and proposes an engineering setpoint. After a successful value-changing write, the rollback manager creates an anchor and an optimization record holding the node, actor, hypothesis, parameter change, and policy, with baseline statistics filled asynchronously from preceding observations. After a five-second same-target deduplication check, a subsequent successful write supersedes the node's open record; an unchanged-value write replaces nothing, while a value-changing agent or rollback write starts a new observation record. Bookkeeping is node-scoped, not one controlled experiment per agent across the plant. Before an agent write, an open record belonging to another agent blocks entry unless superseded as stale, and a recent rollback prevents an agent from immediately moving in the same direction away from the restored value. These checks reduce conflicting interventions but do not exclude manual or recipe-origin writes. Defaults are a 600 s baseline lookback, a 300 s cooldown, and a 30 min stale-record interval.

The agent then invokes `dcw_judge` with a record identifier, verdict, and reason. The tool validates and records the supplied conclusion, so a *keep*, *rollback*, or *uncertain* judgment is attributable reasoning rather than an established process result. A *keep* judgment closes the record and can mark a good state, whereas *rollback* and *uncertain* judgments leave it open; restoring a setpoint requires a separate rollback operation, and uncertainty must not be read as successful optimization.

An independent backstop periodically evaluates eligible open records, waiting at least 120 s by default and rechecking every 30 s. Acquisition streams matching the control node's primary-device alias are preferred, otherwise falling back to the line. Historical query results are compared with configured acquisition monitoring windows, and at least three accumulated breaches trigger a rollback judgment; the queried observations, the aggregation, and data availability all constrain this evidence, and the intervals alone prove no worst-case physical recovery deadline. Under an automatic-rollback policy, the backstop executes recovery only while the node's count of records marked *rolled-back* stays below $K = 2$. That count spans retained records for the node, not the current run alone, and K is not a universal limit on every explicit rollback route. At the threshold, or under an approval-required policy, the backstop records an escalation instead of writing, bounding the implemented backstop decision rather than agent behavior in general.

F. Recipe Management and Rollback

Recipe management separates the definition of intended operation from the device state actually achieved. Versioned

edits and known-good snapshots preserve candidate parameter sets and their provenance, and reverting a definition creates a new version with restored metadata without by itself restoring PLC values. Recipe application instead creates a run record and dispatches its parameters, while line start additionally establishes the active line context; the two should not be conflated. During dispatch, deleted or disabled nodes, and nodes no longer attached to the recipe's line, produce explicit unsuccessful outcomes while the remaining parameters continue, so partial application stays distinguishable from complete. The ordinary active-window check resolves the recipe through the active run's identifier rather than reading every frozen run field; application passes its run identifier into controller writes, skips that branch, and retains runtime range validation, so version history documents intent without establishing transactional consistency among definition, run metadata, and physical parameters.

Physical rollback selects a recorded baseline, anchor, batch-start, or known-good target and issues a new write the controller can reject, so a rollback verdict does not guarantee restoration. Across multiple nodes, recovery is compensating work: one restoration may succeed while another fails, and no distributed physical transaction erases intermediate process states, so operators must distinguish restored metadata, individual compensation, and verified process recovery.

G. Process-Parameter Mapping Layer

A parameter-layer entry projects a control node onto process meaning: it carries a key, a unit, a baseline interval, a product interval and the active recipe window, and it exposes no register, data type or driver-configuration field. A write through this surface is bounded by the intersection of node range, baseline interval, product interval and recipe window, so an agent can reason about a process quantity without seeing a transport address. The layer is an interface over the control path of Sec. IV-C rather than a second gate: it narrows the permitted interval, then enters the same admission checks.

H. Human-in-the-Loop Control

Human intervention attaches to concrete bindings and operations. *Manual* mode suspends execution for approval, while *auto* mode omits that wait but retains the applicable controller and runtime checks. The approval object presented to the operator names the requesting member, node, requested value, effective interval (node range, recipe window, baseline, and product interval intersected), and remaining time. An unbound member is rejected, and after approval the tool verifies that the binding still exists before the request enters the controller. Approval therefore permits a request to proceed; it reserves no device state and guarantees nothing about later checks. Tool approvals expire explicitly; an expired approval executes no write, and the expiry is recorded as a distinct state rather than an operator refusal, so the requester can tell an unanswered approval from a denied one. Duplicate pending requests for the same agent and node are suppressed, and removing a binding can cancel outstanding requests. This timed approval object differs from an optimization record awaiting human action: a

1 Create / connect
Node / protocol
+ 添加节点 modbus- tcp Cast-film line A

2 Device association
Archived node configuration
Process range 150 ~ 200 °C
Bound device Die-head heater

3 Team deployment
Lead / workers lead Cast-film process lead
Archived team definition worker DAQ engineer worker Model trainer

4 Optimization records
Outcome + actor
judged-keep 183 → 178 keep(agent)
superseided 182 → 183 uncertain(system)

Fig. 3. Operator-facing workflow using archived interface crops: node commissioning (the button label is the interface’s original Chinese, “add node”), device association, team deployment, and optimization-record inspection. Device association and agent capability binding are distinct operations, though the interface does not enforce distinct operator roles for them. A displayed judgment records a disposition; it does not independently certify process improvement or physical recovery.

recorded escalation does not mean that recovery has executed, and the case of $K = 2$ being reached while no operator is present is covered by neither path.

Human decisions, agent proposals, and system recovery events provide complementary attribution, and that attribution is useful only while intent, authorization, device outcome, and observed process effect stay distinguishable. Those distinctions are inspectable across the four operator surfaces of Fig. 3. Process-external engines also require deployment-level privilege and network restrictions, because tool bindings govern sanctioned tool calls, not arbitrary I/O by a compromised local process. Section V evaluates these mechanisms against the boundaries marked above.

V. INTEGRATED EVALUATION

A. Evidence Design and the Reference Run

Four questions drive the evaluation: can the platform provision heterogeneous protocol stacks and acquire from them; does the governed write path reject requests that break engineering or recipe constraints without blocking legal ones; can a fixed controller close the acquisition–actuation loop on a process model; and can a scripted team turn a production objective into governed actions that reach it? Source inspection, mock drivers, real protocol transports driving simulated plants, and historical LLM campaigns stay apart; nothing here comes from physical hardware or human operators, and the latency figures do not transfer to physical transports. No headline claim in this paper rests on an LLM decision.

The evidence is not uniform: the out-of-constraint write probes exercise the hard-range and recipe-window checks directly, the lifecycle and mission checks exercise rollback and approval, and the ownership and cooldown checks are traversed by every governed agent write (Sections V-C, V-F, and V-E). The probes are fixed protocol cases, not model-generated adversarial inputs.

Except where an archive is named, the results in this section come from one reference archived run, $B = 20260918043504\text{-bdo}$: seed 42, commit 7d4bfc0, hash 00e7dc827f4a2cea over the modules of the AW-INDUSTRIALBENCH evaluation harness, Node v24.19.0 on an AMD Ryzen 7 9700X host with 31 GB RAM running Windows x64. All 75 checks pass, with no warnings and no failures; the objective, control law, and probe definitions are

fixed properties of the harness source named by that hash, not archive contents. The run therefore measures platform, protocol, governance, controller, and team mechanisms: the agent-tool loops use the deterministic `opcode` engine, and the LLM agent option is off. Three plants are exercised: a cast-film physics model, a strategy-based film line, and a 49-signal biaxially oriented polyethylene terephthalate (BOPET) line. The reference is designated rather than sampled; its position in the eleven-archive family of Sec. V-E is reported there.

B. Suite-Wide Results

Table I groups the suite’s 17 phases into seven evidence layers, all 75 checks passing. The same run stores five vector and five image frames, completes a lead-to-worker dispatch, stores a duplicate memory refresh once, and enumerates 14 registered agent engines, an inventory result that says nothing about control quality across them. The gate’s verdict semantics flag task-level failures as warnings only (Sec. V-F).

Three properties bound the table’s meaning: the 24/24 and 9/9 figures describe configured collision cases, not boundary discrimination; the governance layers exercise specific branches of the write path, not every fault, storage condition, or escalation route; and three checks carry write sub-checks skipped because their nodes expose no writable setpoint, which the pass/warn/fail verdict does not surface.

C. Protocol Integration and Governed Writes

The platform provisions Modbus TCP, OPC UA, MQTT, and HTTP as writable stacks and Modbus RTU over TCP as an acquisition-only satellite. Table II and Fig. 4(a) report write latency and readback; the OPC UA readback entry is a single sample rather than a general claim of exactness.

The 24/24 count lives in phase P3 (per-line integration checks `line-n-io`) and the 9/9 scenario count in phase P8 (checks `port-n-*`). Each writable stack rejects four engineering-range and two recipe-window violations, and none of the four designated legal writes is blocked. The probes are node-high/low $\pm 15\%$ of span, 10^6 , -50 , and the recipe window widened by 20% , so the set shows rejection of gross violations and nothing about where the true limit lies. The readback column rests on one post-write sample per stack, and MQTT and HTTP return no measured value at all; for

TABLE I

INTEGRATED EVALUATION RESULTS IN ARCHIVE *B* (20260918043504-BDO). PHASES ARE THE SUITE'S OWN IDENTIFIERS; "CHECKS" COUNTS PASS/TOTAL WITHIN THE LAYER. THE SEVEN LAYERS PARTITION ALL 17 PHASES AND ALL 75 CHECKS EXACTLY ONCE.

Evidence layer	Phases	Checks	Principal measured result
Plant model and provisioning	P0-P2	9/9	4 writable lines, 1 satellite DAQ; $J^* = 89.894$ as comparison denominator
Protocol I/O and governed write	P3	5/5	56 DAQ samples over 5/5 nodes; 24/24 probes rejected; 0/4 legal writes blocked; p50 mean 33.135 ms (six writes per stack)
Tool-level closed loop	P4	7/7	3 scripted setpoint iterations per line, each accepted and re-observed
Records, approval, recipe, parameter	P4b-P4f	12/12	Verdict separable from compensation; approval in 16 ms; two recipe lifecycles; 4/4 parameter checks
AgentTeam missions	P4m, P10	16/16	One mission reaches $204.704 \pm 0.75^\circ\text{C}$ in 1 write; the biaxial line reaches $25.68\ \mu\text{m}$ from $28.10\ \mu\text{m}$ in 3 writes on 2 protocols
Closed-loop evaluation and scenario reuse	P6, P8, P8b	22/22	$J/J^* = 0.963\text{--}0.972$ over three seeds; film-line 9/9 rejected, 0 false blocks; backstop restores baseline in 120.161 s
Multimodal acquisition and platform services	P7, P9	4/4	5 vector and 5 image frames; dispatch COMPLETED; duplicate memory refresh stored once; 14 engines available
Total	17	75/75	0 warnings, 0 failures; hard gate green

TABLE II

PROTOCOL RESULTS IN *B*. WRITE LATENCY IS p50/p95 OVER SIX TIMED WRITES OF ONE VALUE TO ONE REGISTER PER STACK; WITH SIX SAMPLES THE P95 IS EFFECTIVELY THE OBSERVED MAXIMUM, SO THESE ARE OBSERVATIONS, NOT TAIL ESTIMATES. READBACK IS A SINGLE POST-WRITE SAMPLE; "DAQ" COUNTS STORED DAQ SAMPLES; REJECTION COUNTS SIX AUTHOR-DEFINED OUT-OF-CONSTRAINT PROBES PER WRITABLE STACK. RTU EXPOSES NO WRITABLE SETPOINT. [†]TWO EARLIER WRITES IN THE SAME ARCHIVE DEVIATED BY 0.3–0.4 (THE P4 LINE-2 LOOP).

Stack	DAQ	p50/p95 (ms)	Readback	Reject
Modbus TCP	7	62.717/162.607	0.04 °C	6/6
OPC UA	11	18.711/21.899	exact [†]	6/6
MQTT	11	18.837/19.140	N/A	6/6
HTTP	13	32.273/32.972	N/A	6/6
Modbus RTU over TCP	14	N/A	N/A	N/A

these stacks an accepted write is verified only at the transport level.

The same run provisions the film-line scenario with three writable lines and two satellite nodes: its nine out-of-constraint probes (three node-range probes per writable line) are all rejected and its three legal writes accepted, with no code change; that set exercises no recipe-window branch. Both scenarios come from the same simulator and five drivers, so this is configuration reuse within the existing vocabulary, not portability to an independent plant.

A recorded rollback verdict leaves the register untouched until a separate rollback call passes through the write path; and manual mode suspends an agent request until the scripted approval releases it, measured once at 16ms, timing the approval polling path rather than a human decision. The audit surface holds 11 parameter-change anchors with 200-entry audit and operations-log windows, so intent, approval, and execution are separately inspectable, though the journal is not a signed ledger and protocol I/O is not atomic with a concurrent recipe change.

A frozen four-arm ablation, archived separately (run-ellipse-4arm in the frozen 20260914 baseline; seed 42, three repetitions, commit d6c824d, an ancestor of the reference commit), shows what the write path contributes; its arms are full, no interlock, no readback, and ungated. The ablation runs on the mock driver tier against a single-node recipe window, and its three repetitions per arm produce

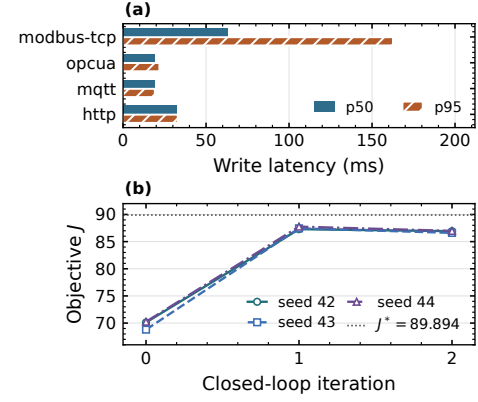


Fig. 4. Archive *B*. (a) Governed write latency p50 and p95 for the four writable stacks (numeric values in Table II); Modbus TCP carries the longest tail. (b) Fixed-controller objective J for seeds 42–44, sampled at the start and after each of two governed writes, against the archived finite-grid reference $J^* = 89.894$. J is maximized. Each marker is one measured evaluation; the third lies below the second because the second write raises defect rate. The archived J_{end} (mean of the last two evaluations) is a separate statistic and is not plotted.

identical verdicts (latencies and a few probe readings vary at sub-check resolution), i.e., the ablation evidence is one deterministic decision per arm. Per-write latency showed no detectable difference at the observed resolution. With the full path, all six probes are rejected. With the batch-window interlock disabled, or with the path ungated, four are still rejected and the two recipe-window violations execute at the (mock) driver tier; disabling readback alone changes nothing. The engineering range is therefore structurally enforced at the mock tier, and the batch-window check is the one that adds protection, while attribution records what happened without preventing it. The ablation archive carries boundary probes one tenth of a unit either side of the window edge: rejected where the window check is active, accepted where it is disabled, with no false block in any arm. The process-parameter layer passes 4/4 checks: out-of-product and out-of-baseline values are rejected at their respective layers, the parameter reads back, and an unbound agent is denied; agents thus receive a process-facing interface that does not disclose transport addresses (Sec. IV-G).

D. Fixed-Controller Closed-Loop Benchmark

The controller in this layer contains no LLM. It starts the cast-film twin at screw speed $N = 150$ rpm, line speed $v = 95$ m/min, and zone setpoint 200°C , and *maximizes* the objective

$$J = 55 J_{\text{thick}} + 25 J_{\text{quality}} + 8 J_{\text{energy}} + 7 J_{\text{throughput}}, \quad (3)$$

subject to melt temperature in $[195, 225]^\circ\text{C}$ and pressure at most 22 MPa. Each term is a normalized score in $[0, 1]$, so $J \in [0, 95]$: thickness quality saturates once $|h - 50| \leq 2 \mu\text{m}$, and the other terms reward low defect rate, low screw speed, and high line speed. The controller updates $N \leftarrow N(50/h)$ under a 22-rpm step limit, holds line speed fixed, and corrects heating only when temperature leaves $[196, 224]^\circ\text{C}$. The reference J^* comes from the separately maintained simulator, not from the run. Two features of that scoring bound the ratio: the throughput and energy terms are evaluated from the issued setpoints rather than from measured values, because the twin returns no independent reading for them, and the control law inverts the model's own relation $h \propto N/v$, so the plateau is structural. The suite's gate is $J/J^* \geq 0.8$, so the reported value is not near a threshold.

The three seeds start at $J_0 = 70.115, 68.792$, and 70.248 ; Fig. 4(b) plots each seed's objective against the archived reference. All six governed writes are accepted and the mean wall time is 36.193 s per seed. The archive reports an end-of-run score $J_{\text{end}} = 87.105, 87.027$, and 87.366 (the mean of the last two evaluations), giving $J_{\text{end}}/J^* = 0.969, 0.968$, and 0.972 ; the last measured evaluation is lower in every seed, at 86.921, 86.609, and 86.975, or 0.967, 0.963, and 0.968. The stopping rule requires two consecutive converged observations, and the first write already brought thickness inside the $50 \pm 2 \mu\text{m}$ plateau, so the second write follows from the rule and not from further improvement: it raises defect rate from 0.38–0.53 % to 0.62–0.72 % and lowers the composite objective. We therefore quote 0.963–0.972 across both conventions and do not describe the trajectory as monotone. Final thicknesses are 49.883, 49.383, and $49.317 \mu\text{m}$.

Decomposing J at the final point shows the residual gap: both reference and seeds saturate the thickness term, and the seeds' deficit on defect quality, screw-speed energy, and line throughput reflects the controller's restricted action set (it holds line speed at 95 m/min while the reference reaches $h = 49.18 \mu\text{m}$ at 40 m/min and 55 rpm), not a failure to reach thickness.

E. Scripted Team Missions

Both missions are dispatched by *scripted* channel members: the harness creates the lead and the worker on its mock engine and the benchmark script issues every tool call itself, so no language model decides a setpoint; the missions exercise the path an agent would use, not agent performance. The first asks whether an objective filed on the channel task board can travel through team delegation, a governed write, and back to a closed task: bring a Modbus-TCP temperature node to $204.704 \pm 0.75^\circ\text{C}$ in at most three governed writes after reading recent data. The lead dispatches the task, a bound worker

reads five minutes of daq history, and one `dcw_control` request raises the setpoint from 200 to 204.704; the readback returns 204.700, the record is judged `keep`, the write is journal-attributed, and the parent task reaches `COMPLETED` with all six mission checks passing. The $\pm 0.75^\circ\text{C}$ band is loose (a 204.0°C request would also pass); the mission shows the delegation–write–completion path closes, not that the band certifies precise targeting.

The second mission is the more demanding one; Fig. 5 maps it to the line, the team, the admission gate, the three recorded writes, and the response they produced. Its BOPET preset is provisioned as one full line with nine devices, 30 writable setpoints, 19 configured acquisition nodes, and driver tests passing on all nine, all from the benchmark's own probe-and-provision step, so this layer measures provisioning and governance, not commissioning a line that already existed. The scripted team files a thickness objective of $25.0 \pm 0.7 \mu\text{m}$, accepted as $|h - 25.0| \leq 0.7 \mu\text{m}$; the initial gauge reading through the real MQTT path is $28.10 \mu\text{m}$, outside the window. The worker then applies three governed actions: cast-roll speed $32 \rightarrow 33.8$ m/min on the casting Modbus-TCP node brings thickness $28.10 \rightarrow 26.62 \mu\text{m}$ (`opt-86313140`); fast-roll speed $118 \rightarrow 120.5$ m/min on the machine-direction orienter (MDO) Modbus-TCP node brings it to $25.90 \mu\text{m}$ (`opt-41929fe2`); and exit rail width $3000 \rightarrow 3031$ mm on the transverse-direction orienter (TDO) OPC UA node brings it to $25.68 \mu\text{m}$ (`opt-8d2faadb`). Each action is a separate governed write to a different line section, spanning two protocols and two engineering units. Every write cleared the gate of Fig. 5 (checks `biacx-mission-multinode` and `biacx-mission-attained`), was accepted by its driver, opened its own optimization record, and was followed by a gauge re-read after a settling wait; the archive stores no post-write readback value for these three writes, so we claim none. The verdicts are not an independent finding: the scripted caller issues `keep` unconditionally, so the records show that judgment is stored separately from execution and compensation, not that a judgment was earned. The journal query for the cast-speed node returned an agent-source record (the parameter layer of Sec. V-C); the mission check asserting journal attribution is the one with an empty evidence array in Sec. V-F.

Eleven archived runs of the reference command family share its seed; ten match its command line verbatim and one collapsed archive adds an LLM-agent flag. Seven report a final thickness inside the band (a small- n observation, not a reliability estimate), three collapsed to near-zero thickness and reappear in the film-break account of Sec. V-F, and one registered zero governed writes with a hard multinode failure and no final thickness, excluding it from the ten with reportable endpoints. The reruns span several harness revisions; the strictly same-version comparison is the $25.68 \mu\text{m} / -0.02 \mu\text{m}$ pair of Sec. V-F. The run reported here is one of the seven.

The final thickness is $25.68 \mu\text{m}$, satisfying $|h - 25.0| = 0.68 \leq 0.7 \mu\text{m}$ by $0.02 \mu\text{m}$ on a short bucket mean (the mean of the last five one-second acquisition buckets), and the mission closes with 3 of its 6 allowed writes used. This margin is finer than the run-to-run variation of Sec. V-F and

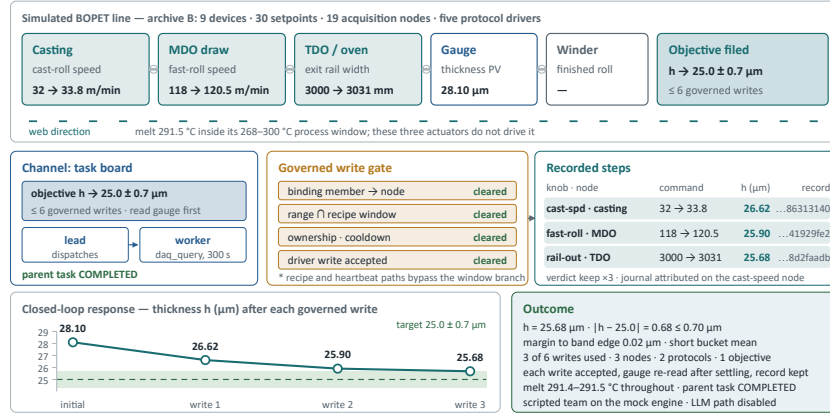


Fig. 5. Scripted AgentTeam mission in archive *B*, reconstructed from *agentteam-biax.log* and *run.json*. Top: the simulated BOPET line (nine devices, 30 setpoints, 19 acquisition nodes) with the three written actuators, the gauge, and the objective. Middle: the channel carrying the objective, the four checks a governed write clears (the first three gate admission; the fourth is the driver’s acceptance), and the three recorded steps (of an allowed six). Bottom: thickness after each step against the $25.0 \pm 0.7 \mu\text{m}$ objective; the final point clears the band edge by $0.02 \mu\text{m}$. Team members are scripted mock agents with a prespecified proportional rule; the LLM agent stays disabled, so the figure documents the governance and attribution path rather than optimization quality.

characterizes this run, not a robust property. Melt temperature stays near 291.5°C inside its $268\text{--}300^\circ\text{C}$ process window, an observation rather than an enforcement result, since no disturbance tested that bound.

The mission is a scripted policy, not a search: each actuator carries a preset proportional gain on the relative thickness error, chosen by fixed rotation through the three knobs, with gains 0.5, 0.35, and 0.3 set deliberately low so convergence takes three to four rounds and more than one node is involved; the shrinking response (1.48 , 0.72 , and $0.22 \mu\text{m}$) and the three-write length follow from that configuration, not from emergence. What the mission shows is operational: the binding model resolves a different node, range, unit, and recipe window for each write, and the records keep the three writes separate while attributing them to one objective; it does not establish autonomous strategy quality, cross-model generality, or physical-process performance.

F. Reproducibility, Recovery, and Threats to Validity

The backstop drill freezes a process variable outside its active window; the harness injects the breach rather than a drifting plant producing it. The system identifies an eligible open record, records a rollback judgment, and restores the recorded baseline in 120.161 s , a latency set by the backstop’s own wait-then-recheck cadence rather than by the plant. The drill shows one bounded recovery execution: it does not exercise escalation at $K = 2$, does not test detection of a slowly drifting excursion or of a gap between batches, and sets no worst-case recovery deadline.

First, during harness development a burst of writes across several protocols drove the plant into a collapsed state—pump pressure, draw ratio, and thickness at their limits—while every platform-side write stayed readback-consistent: a readback therefore confirms that a driver delivered a value, not that the plant is in a sane state. Second, a film-break guard that reapplies the recipe baseline is present in the mission harness but is not represented in archive *B*, which never entered that branch. Three archives of the Sec. V-E

family entered it, one of them earlier on the same day at the same recorded commit, and in each case the re-application failed to restore thickness, leaving the plant near or below zero thickness (final values 0.02 , -0.02 , $-0.02 \mu\text{m}$ against a $\sim 25 \mu\text{m}$ band); one $-0.02 \mu\text{m}$ archive is the same-hash, same-seed sister of the nondeterminism example below. A fourth archive collapsed without entering the branch. The guard therefore aborts rather than recovers, and end states near or below zero thickness indicate that the plant model, not only the governance layer, bounds this result. In each of the three collapses the mission’s own optimization records were superseded within about a minute of the write, so the independent backstop’s 120-second eligibility window never opened; the guard’s abort, not a backstop rollback, is what ended those runs. Third, the audit path fails open: bookkeeping errors are caught without reversing an executed write, so a physical change can occur without an attribution record, and the platform logs the error without blocking the write or retrying the record. Attribution is therefore a strong default, not a guarantee.

The command `(node bench/pipeline.mjs --profile integrated --seed 42 --cl-seeds 3)`, seed, commit, and a hash over the nine harness modules let another operator rerun the suite and compare judge-class outcomes against the same baseline. Five caveats remain. (1) The harness-hash covers working-tree modules and is sensitive to checkout line endings. (2) The plant simulator is a separately versioned dependency outside the fingerprint, so the exact harness and plant behind these numbers cannot be reconstructed from the archive alone. (3) The biaxial journal-attribution check passed with an empty evidence array, so the run bookkeeping is not uniformly clean. (4) The checks were written against this implementation. (5) The reference J^* comes from the same simulator the controller optimises, so the gate is a development-time regression suite, not a benchmark with an external oracle. The simulator snapshot submitted with the paper corresponds to the plant state described here. A final caveat concerns the mission

layer. Its starting thickness is a simulator residue rather than a pinned initial condition. Later runs of the same command on the same host reached different outcomes: two archives sharing one harness hash and one seed, neither of them B , ended at $25.68\ \mu\text{m}$ and at $-0.02\ \mu\text{m}$. In the second, both mission checks are recorded as warnings, no check failed, and the hard gate stayed green. The suite is therefore not deterministic at the mission layer, and the 75/75 gate describes archive B rather than a property of the system. Whether this stems from harness-version drift or runtime nondeterminism was not isolated; a rerun campaign against a pinned simulator revision is required to separate the two. Verdicts and record states are deterministic within a run; protocol latencies are not, and the two should not be read as interchangeable. The evidence remains simulation-tier: no physical device, TEP or SWaT replay, human-operator workload study, or external-framework comparison was run.

Historical real-LLM campaigns are exploratory only: same governed interface but different scoring inputs and temporal windows, a legacy script that substituted missing observations, and a completion regex that could match an instruction template—so their scores are not aggregated above. Archive B uses the scripted path; Section VI collects the resulting limitations.

VI. DISCUSSION AND LIMITATIONS

A. Established Properties and Their Boundaries

The reusable unit here is the deployed member–node capability–production–run association: it lets one task work in engineering quantities across drivers while execution and compensation stay attributable, and let the biaxial mission move three line sections under one objective without plant-specific routing code. It does not measure the effort to author a scenario or integrate a protocol the platform does not already speak.

Proposal, admission, and disposition stay distinguishable: a recorded `keep` is the agent’s judgment, a matching readback is a driver-level observation, and sustained process convergence is a separate experimental outcome. Observation-quality gating is a framework-level extension this paper does not deliver; until it exists, the historical LLM scores cannot be compared with the deterministic ones: their writes executed as designed, but their judgments were formed over scoring inputs and temporal windows that diverged from the plant’s actual state (Sec. V-F).

B. Implementation and Evidence Boundaries

Cross-agent in-window oscillations, shared-node contention over longer horizons, and availability during storage failure remain unevaluated. The collapse archives of Sec. V-E also expose a suppression mode: supersession within about a minute kept the backstop’s 120-second eligibility window from opening, so sustained rapid writes defer independent rollback indefinitely; a minimum record-dwell before supersession is a candidate mitigation. Approval reduces permitted automation without establishing endpoint security, protecting against a

privileged compromised harness, or validating approved engineering intent; least-privilege deployment and network separation remain separate responsibilities [40], [41]. Governed writes are defense-in-depth at the basic process control system layer, not a certified safety instrumented function: their safety argument presupposes independent regulatory limits and a protection layer in the IEC 61511 sense [42], and the platform implements no detection or degraded-mode response when such layers are absent, which is an integrator responsibility. Because the governed path rejects out-of-constraint writes, a protection-layer analysis must likewise not record it as an independent protection layer without the assessment IEC 61511 assigns to such layers. Manual-binding approval workload grows with the frequency of agent proposals, and approval fatigue and over-reliance are documented risks [31], [32], [43], [44]; policy granularity over roles, batch phases, and magnitudes is an extension point, not a present capability, and segregation of duties between device association and capability binding is not enforced by the current interface (Fig. 3). None of these mechanisms is enforced outside the platform process, so they constrain only traffic that passes through it; a deployment therefore needs independent setpoint limits and interlocks in the controller itself.

Deployment in a real plant carries prerequisites this paper does not discharge: placement across the IT/OT hierarchy and IEC 62443 zones-and-conduits assignment are integrator decisions [40]. A deployment design must also answer the platform’s failure modes, including the approval channel, alarm paths [45], [46], and open records left by a core crash, and must supply clock synchronization, off-host record preservation, and pinned engine versions where reproducible behavior is required. The single-host Windows capacity boundary is likewise untested, as is splitting the co-resident deployment across a 62443 zone boundary with the approval channel as the only conduit.

VII. CONCLUSION

AGENTWORKSHOP integrates multi-agent coordination with heterogeneous industrial supervision through explicit node capabilities and shared production context, keeping coordination and supervision cooperating without becoming one atomic control function. Reference archive B provides three complementary results: four writable stacks reject all predefined out-of-constraint writes with no legal-write blocks; a fixed controller reaches $J/J^* = 0.963\text{--}0.972$ across three seeds; and a scripted team mission uses three governed writes across two protocols to bring biaxial-film thickness from 28.10 to $25.68\ \mu\text{m}$. Seven of the ten reportable reruns (of eleven same-seed archives, including the reference) attain the band, the reported run among them; the failed reruns end near zero thickness with no check failing and the mission checks downgraded to warnings, and the suite’s hard gate does not flag them. No LLM client has yet exercised the governed path under controlled evaluation; the historical campaigns of Sec. V-F remain exploratory. These results demonstrate the tested integration and governance paths only; physical validation, stronger execution consistency, controlled agent

comparisons, observation-quality gating, and evaluation of the attribution records by downstream investigators remain open.

DATA AVAILABILITY AND DISCLOSURE

The archive 20260918043504-bdo used in Sec. V is machine-readable—`run.json` (every check with evidence), `summary.json` (verdict, KPIs), `metrics.csv` (metric registry), `agentteam-*.log` (mission traces)—and ships with the figure scripts and the archive-consistency checker. Anonymized platform, benchmark-card, and simulator snapshots accompany the submission; repository identifiers will be disclosed on acceptance. AI-based assistants supported manuscript editing, evidence checking, and figure preparation under author direction; the authors remain responsible for the claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Sci. Rep.*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Comput. Ind.*, vol. 162, 2024, art. no. 104129.
- [3] Z. e. a. Liu, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakih, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 979–991, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IGK: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Trans. Ind. Inform.*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, arXiv preprint arXiv:2405.18092.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, arXiv preprint arXiv:2505.02076.
- [12] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-llm-in-the-loop framework for context-aware control," *arXiv preprint arXiv:2504.05946*, 2025.
- [13] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.
- [14] Z. e. a. Zhao, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.
- [15] Q. e. a. Wu, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [16] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024.
- [17] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024.
- [18] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024.
- [19] Linux Foundation, "Agent2agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025.
- [20] International Society of Automation, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.
- [21] —, "ANSI/ISA-88.01:2010—batch control part 1: Models and terminology (international equivalent IEC 61512-1)," Standard, 2010.
- [22] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Inform.*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [23] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [24] International Electrotechnical Commission, "IEC 63278 (series): Asset administration shell for industrial applications," International Standard, 2023.
- [25] A. Gautam *et al.*, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, 2025.
- [26] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.
- [27] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.
- [28] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [29] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [30] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.
- [31] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.
- [32] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [33] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [34] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [35] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.
- [36] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [37] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
- [38] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. USENIX Security Symposium*, 2024.
- [39] International Electrotechnical Commission, "IEC 62541: OPC unified architecture," International Standard, 2020.
- [40] —, "IEC 62443: Security of industrial automation and control systems," International Standard, 2018.
- [41] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [42] International Electrotechnical Commission, "IEC 61511: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.
- [43] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [44] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
- [45] International Society of Automation, "ANSI/ISA-18.2: Management of alarm systems for the process industries (iec 62682)," Standard, 2016.
- [46] Engineering Equipment and Materials Users Association, "Eemua 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.